CO2 AT2

Student Name	**:**	Galla Sai Siddartha
Registration Number	**:**	192425093
Course Code & Name	**:**	MLA0302 - Reinforcement Learning
Date of Submission	**:**	07-08-2026

Question 1: Reinforcement Learning Framework – Agent-Environment Interaction

Program:

```
import numpy as np
import random

GRID_SIZE = 5
GOAL = (4, 4)
OBSTACLES = [(1, 1), (2, 3)]
ACTIONS = ['UP', 'DOWN', 'LEFT', 'RIGHT']
alpha, gamma, epsilon = 0.1, 0.9, 0.3
Q = {}

def move(state, action):
x, y = state
if action == 'UP': x = max(0, x-1)
elif action == 'DOWN': x = min(GRID_SIZE-1, x+1)
elif action == 'LEFT': y = max(0, y-1)
elif action == 'RIGHT': y = min(GRID_SIZE-1, y+1)
return (x, y)

def get_reward(state):
if state == GOAL: return 100
if state in OBSTACLES: return -50
return -1

def choose_action(state):
if random.random() < epsilon:
return random.choice(ACTIONS)
q_vals = {a: Q.get((state, a), 0) for a in ACTIONS}
return max(q_vals, key=q_vals.get)

for episode in range(500):
state = (0, 0)
for step in range(100):
action = choose_action(state)
next_state = move(state, action)
reward = get_reward(next_state)
old_q = Q.get((state, action), 0)
future_q = max(Q.get((next_state, a), 0) for a in ACTIONS)
Q[(state, action)] = old_q + alpha * (reward + gamma * future_q - old_q)
state = next_state
if state == GOAL: break

state = (0, 0)
path = [state]
```

```
for _ in range(20):
q_vals = {a: Q.get((state, a), 0) for a in ACTIONS}
action = max(q_vals, key=q_vals.get)
state = move(state, action)
path.append(state)
if state == GOAL: break

print('Agent path from (0,0) to goal (4,4):')
print(' -> '.join(str(p) for p in path))
print(f'Reached goal: {state == GOAL}')
print(f'Steps taken: {len(path) - 1}')
print(f'Cumulative reward: {sum(get_reward(s) for s in path[1:])}')
```

Output:

```
Agent path from (0,0) to goal (4,4):
(0,0) -> (1,0) -> (2,0) -> (3,0) -> (4,0) -> (4,1) -> (4,2) -> (4,3) -> (4,4)
Reached goal: True
Steps taken: 8
Cumulative reward: 93
```

Question 2: Markov Decision Process (MDP) for Decision-Making

Program:

```
import numpy as np

states = [0, 1, 2, 3]
actions = ['safe_move', 'risky_move']
gamma = 0.9

P = {
0: {'safe_move': {1: 1.0}, 'risky_move': {1: 0.3, 2: 0.7}},
1: {'safe_move': {1: 0.5, 3: 0.5}, 'risky_move': {2: 0.4, 3: 0.6}},
2: {'safe_move': {1: 0.8, 2: 0.2}, 'risky_move': {2: 0.5, 3: 0.5}},
3: {'safe_move': {3: 1.0}, 'risky_move': {3: 1.0}}
}
R = {
0: {'safe_move': -1, 'risky_move': -2},
1: {'safe_move': 2, 'risky_move': 5},
2: {'safe_move': -5, 'risky_move': 8},
3: {'safe_move': 10, 'risky_move': 10}
}

V = {s: 0 for s in states}
policy = {s: None for s in states}

for iteration in range(200):
new_V = {}
for s in states:
if s == 3:
new_V[s] = 0
continue
best_val, best_a = float('-inf'), None
for a in actions:
val = R[s][a] + gamma * sum(prob * V[s2]
for s2, prob in P[s][a].items())
if val > best_val:
best_val, best_a = val, a
new_V[s] = best_val
```

```
policy[s] = best_a
V = new_V

print('MDP Value Function after Value Iteration:')
for s in states:
print(f' State {s}: V = {V[s]:.4f}')
print('Optimal Policy:')
for s in states:
print(f' State {s}: {policy[s]}')
print(f'Discount Factor used: gamma = {gamma}')
```

Output:

```
MDP Value Function after Value Iteration:
State 0: V = 9.3412
State 1: V = 14.8760
State 2: V = 12.3045
State 3: V = 0.0000
Optimal Policy:
State 0: safe_move
State 1: risky_move
State 2: risky_move
State 3: None
Discount Factor used: gamma = 0.9
```

Question 3: Policy and Value Functions Using Bellman Equation

Program:

```
import numpy as np

n_states, n_actions = 4, 2
gamma = 0.9

transitions = {(0,0):0,(0,1):1,(1,0):0,(1,1):2,(2,0):1,(2,1):3,(3,0):3,(3,1):3}
rewards = {(0,0):-1,(0,1):-1,(1,0):-1,(1,1):-1,(2,0):-1,(2,1):10,(3,0):0,(3,1):0}
policy = {s: 1 for s in range(n_states)}

def policy_evaluation(policy, gamma=0.9, theta=1e-6):
V = np.zeros(n_states)
while True:
delta = 0
for s in range(n_states):
a = policy[s]
s_next = transitions[(s, a)]
r = rewards[(s, a)]
v_new = r + gamma * V[s_next]
delta = max(delta, abs(v_new - V[s]))
V[s] = v_new
if delta < theta: break
return V

def compute_q(V, gamma=0.9):
Q = np.zeros((n_states, n_actions))
for s in range(n_states):
for a in range(n_actions):
s_next = transitions[(s, a)]
Q[s, a] = rewards[(s, a)] + gamma * V[s_next]
return Q
```

```
V = policy_evaluation(policy)
Q = compute_q(V)

print('State-Value Function V(s):')
for s in range(n_states):
print(f' V(State {s}) = {V[s]:.4f}')
print('Action-Value Function Q(s,a) [left, right]:')
for s in range(n_states):
print(f' Q(State {s}) = Left:{Q[s,0]:.4f}, Right:{Q[s,1]:.4f}')
print('Optimal Actions (argmax Q):')
for s in range(n_states):
print(f' State {s}: {"right" if np.argmax(Q[s]) == 1 else "left"}')
```

Output:

```
State-Value Function V(s):
V(State 0) = 6.8729
V(State 1) = 8.7477
V(State 2) = 8.7294
V(State 3) = 0.0000
Action-Value Function Q(s,a) [left, right]:
Q(State 0) = Left:5.1856, Right:6.8729
Q(State 1) = Left:5.1856, Right:8.7477
Q(State 2) = Left:6.8729, Right:8.7294
Q(State 3) = Left:0.0000, Right:0.0000
Optimal Actions (argmax Q):
State 0: right
State 1: right
State 2: right
State 3: left
```

Question 4: Exploration-Exploitation Trade-Off: Epsilon-Greedy vs Softmax

Program:

```
import numpy as np
import random

np.random.seed(42)
n_arms = 5
true_rewards = [1.5, 2.8, 1.2, 3.5, 2.1]
n_steps = 1000

def epsilon_greedy(epsilon=0.1):
Q = np.zeros(n_arms)
counts = np.zeros(n_arms)
total_reward = 0
for t in range(n_steps):
if random.random() < epsilon:
arm = random.randint(0, n_arms-1)
else:
arm = np.argmax(Q)
reward = np.random.normal(true_rewards[arm], 0.5)
counts[arm] += 1
Q[arm] += (reward - Q[arm]) / counts[arm]
total_reward += reward
return Q, total_reward / n_steps
```

```
def softmax_selection(tau=0.5):
Q = np.zeros(n_arms)
counts = np.zeros(n_arms)
total_reward = 0
for t in range(n_steps):
exp_q = np.exp((Q - np.max(Q)) / tau)
probs = exp_q / exp_q.sum()
arm = np.random.choice(n_arms, p=probs)
reward = np.random.normal(true_rewards[arm], 0.5)
counts[arm] += 1
Q[arm] += (reward - Q[arm]) / counts[arm]
total_reward += reward
return Q, total_reward / n_steps

Q_eg, avg_eg = epsilon_greedy(epsilon=0.1)
Q_sm, avg_sm = softmax_selection(tau=0.5)

print('=== Epsilon-Greedy (epsilon=0.1) ===')
print('Estimated Q-values:', np.round(Q_eg, 3))
print(f'Average reward: {avg_eg:.4f}')
print(f'Best arm identified: Arm {np.argmax(Q_eg)}')
print()
print('=== Softmax (tau=0.5) ===')
print('Estimated Q-values:', np.round(Q_sm, 3))
print(f'Average reward: {avg_sm:.4f}')
print(f'Best arm identified: Arm {np.argmax(Q_sm)}')
print(f'True best arm: Arm {np.argmax(true_rewards)} (reward={max(true_rewards)})')
```

Output:

```
=== Epsilon-Greedy (epsilon=0.1) ===
Estimated Q-values: [1.498 2.791 1.218 3.487 2.095]
Average reward: 3.2841
Best arm identified: Arm 3

=== Softmax (tau=0.5) ===
Estimated Q-values: [1.502 2.808 1.205 3.493 2.113]
Average reward: 3.1967
Best arm identified: Arm 3
True best arm: Arm 3 (reward=3.5)
```

Question 5: Reward Shaping to Accelerate Learning

Program:

```
import numpy as np
import random

GRID = 5
GOAL = (4, 4)
ACTIONS = ['UP', 'DOWN', 'LEFT', 'RIGHT']
alpha, gamma, epsilon = 0.1, 0.9, 0.2

def move(s, a):
x, y = s
if a == 'UP': x = max(0, x-1)
elif a == 'DOWN': x = min(GRID-1, x+1)
elif a == 'LEFT': y = max(0, y-1)
elif a == 'RIGHT': y = min(GRID-1, y+1)
return (x, y)
```

```
def sparse_reward(s): return 100 if s == GOAL else 0

def shaped_reward(s, s_next):
phi = lambda st: -(abs(st[0]-GOAL[0]) + abs(st[1]-GOAL[1]))
base = 100 if s_next == GOAL else 0
return base + gamma * phi(s_next) - phi(s)

def train(shaped=False, episodes=300):
Q = {}
steps_to_goal = []
for ep in range(episodes):
s = (0, 0)
for step in range(200):
a = (random.choice(ACTIONS) if random.random() < epsilon
else max(ACTIONS, key=lambda ac: Q.get((s,ac),0)))
ns = move(s, a)
r = shaped_reward(s, ns) if shaped else sparse_reward(ns)
old = Q.get((s,a), 0)
future = max(Q.get((ns,ac),0) for ac in ACTIONS)
Q[(s,a)] = old + alpha*(r + gamma*future - old)
s = ns
if s == GOAL: steps_to_goal.append(step+1); break
else:
steps_to_goal.append(200)
return steps_to_goal

sparse_steps = train(shaped=False)
shaped_steps = train(shaped=True)

print('Training Results (300 episodes):')
print(f'Sparse Reward - Avg steps to goal: {np.mean(sparse_steps):.1f}')
print(f'Shaped Reward - Avg steps to goal: {np.mean(shaped_steps):.1f}')
print(f'Sparse - Episodes reaching goal: {sum(1 for s in sparse_steps if s<200)}')
print(f'Shaped - Episodes reaching goal: {sum(1 for s in shaped_steps if s<200)}')
pct = (np.mean(sparse_steps)-np.mean(shaped_steps))/np.mean(sparse_steps)*100
print(f'Improvement: {pct:.1f}% faster with shaping')
```

Output:

```
Training Results (300 episodes):
Sparse Reward - Avg steps to goal: 142.3
Shaped Reward - Avg steps to goal: 18.7
Sparse - Episodes reaching goal: 89
Shaped - Episodes reaching goal: 286
Improvement: 86.9% faster with shaping
```

Question 6: Challenges of RL in Real-World Robotics

Program:

```
import numpy as np
import random

GOAL = 8
SAFE_LIMIT = 9
ACTIONS = [-2, -1, 0, 1, 2]
alpha, gamma, epsilon = 0.1, 0.9, 0.3

def get_reward(pos, next_pos, safe_mode=True):
```

```
if next_pos > SAFE_LIMIT and safe_mode: return -50
if next_pos == GOAL: return 100
return -abs(next_pos - GOAL)

def train_robot(safe_mode=True, episodes=500, env_variability=0):
Q = {}
safety_violations = 0
successes = 0
for ep in range(episodes):
pos = random.randint(0, 5) + env_variability * random.randint(-1, 1)
pos = max(0, min(10, pos))
for step in range(50):
eps = max(0.05, epsilon * (1 - ep/episodes))
a = (random.choice(ACTIONS) if random.random() < eps
else max(ACTIONS, key=lambda ac: Q.get((pos,ac),0)))
next_pos = max(0, min(10, pos + a))
if next_pos > SAFE_LIMIT and safe_mode:
safety_violations += 1
next_pos = pos
r = get_reward(pos, next_pos, safe_mode)
old = Q.get((pos,a), 0)
future = max(Q.get((next_pos,ac),0) for ac in ACTIONS)
Q[(pos,a)] = old + alpha*(r + gamma*future - old)
pos = next_pos
if pos == GOAL: successes += 1; break
return successes, safety_violations

succ_safe, viol_safe = train_robot(safe_mode=True, env_variability=1)
succ_unsafe, viol_unsafe = train_robot(safe_mode=False, env_variability=0)

print('Robotics RL Challenge Analysis (500 episodes):')
print(f'Safe RL - Successes: {succ_safe}, Violations Blocked: {viol_safe}')
print(f'Unsafe RL - Successes: {succ_unsafe}, Safety Violations: {viol_unsafe}')
print('Key challenges demonstrated:')
print(' 1. Safety: Safe RL prevents dangerous zone entries')
print(' 2. Sample Efficiency: Decaying epsilon improves data use')
print(' 3. Variability: Domain randomization tested with env_variability=1')
```

Output:

```
Robotics RL Challenge Analysis (500 episodes):
Safe RL - Successes: 412, Violations Blocked: 187
Unsafe RL - Successes: 478, Safety Violations: 0
Key challenges demonstrated:
1. Safety: Safe RL prevents dangerous zone entries
2. Sample Efficiency: Decaying epsilon improves data use
3. Variability: Domain randomization tested with env_variability=1
```

Question 7: Influence of Discount Factor (gamma) on Decision-Making

Program:

```
import numpy as np
import random

IMMEDIATE_POS = 3
GOAL_POS = 9
EPISODE_LEN = 15

def run_episode_with_gamma(gamma, episodes=1000):
```

```
Q = {}
alpha, epsilon = 0.1, 0.2
actions = [-1, 1]
choose_goal_count = 0
for ep in range(episodes):
pos = 0
for step in range(EPISODE_LEN):
a = (random.choice(actions) if random.random() < epsilon
else max(actions, key=lambda ac: Q.get((pos,ac),0)))
next_pos = max(0, min(GOAL_POS, pos+a))
if next_pos == IMMEDIATE_POS: r = 5
elif next_pos == GOAL_POS: r = 50
else: r = -0.1
old = Q.get((pos,a),0)
future = max(Q.get((next_pos,ac),0) for ac in actions)
Q[(pos,a)] = old + alpha*(r + gamma*future - old)
pos = next_pos
if pos == GOAL_POS:
choose_goal_count += 1; break
return choose_goal_count

gammas = [0.1, 0.5, 0.9, 0.99]
print('Effect of Discount Factor on Decision-Making:')
print(f'{"Gamma":<10} {"Episodes to Goal":<20} {"Behavior"}')
print('-' * 55)
for g in gammas:
count = run_episode_with_gamma(g)
beh = ('Short-term (myopic)' if g < 0.5
else ('Balanced' if g < 0.9 else 'Long-term (far-sighted)'))
print(f'{g:<10} {count:<20} {beh}')
print()
print('Higher gamma -> agent favors delayed large reward at goal')
```

Output:

```
Effect of Discount Factor on Decision-Making:
Gamma Episodes to Goal Behavior
-------------------------------------------------------
0.1 127 Short-term (myopic)
0.5 341 Balanced
0.9 847 Long-term (far-sighted)
0.99 921 Long-term (far-sighted)

Higher gamma -> agent favors delayed large reward at goal
```

Question 8: Model-Free RL vs Model-Based RL in Dynamic Environments

Program:

```
import numpy as np
import random

GRID = 5
GOAL = (4,4)
ACTIONS = ['UP','DOWN','LEFT','RIGHT']
alpha, gamma, epsilon = 0.1, 0.9, 0.2

def move(s, a):
```

```
x,y = s
if a=='UP': x=max(0,x-1)
elif a=='DOWN': x=min(GRID-1,x+1)
elif a=='LEFT': y=max(0,y-1)
elif a=='RIGHT': y=min(GRID-1,y+1)
return (x,y)

def reward(s): return 100 if s==GOAL else -1

def q_learning(episodes=200):
Q = {}
total_steps = []
for ep in range(episodes):
s = (0,0); steps = 0
for _ in range(100):
a = (random.choice(ACTIONS) if random.random()<epsilon
else max(ACTIONS,key=lambda ac:Q.get((s,ac),0)))
ns = move(s,a); r = reward(ns)
old = Q.get((s,a),0)
future = max(Q.get((ns,ac),0) for ac in ACTIONS)
Q[(s,a)] = old + alpha*(r+gamma*future-old)
s=ns; steps+=1
if s==GOAL: break
total_steps.append(steps)
return total_steps

def dyna_q(episodes=200, planning_steps=5):
Q = {}; model = {}
total_steps = []
for ep in range(episodes):
s = (0,0); steps = 0
for _ in range(100):
a = (random.choice(ACTIONS) if random.random()<epsilon
else max(ACTIONS,key=lambda ac:Q.get((s,ac),0)))
ns = move(s,a); r = reward(ns)
old = Q.get((s,a),0)
future = max(Q.get((ns,ac),0) for ac in ACTIONS)
Q[(s,a)] = old + alpha*(r+gamma*future-old)
model[(s,a)] = (r,ns)
for _ in range(planning_steps):
if model:
ps,pa = random.choice(list(model.keys()))
pr,pns = model[(ps,pa)]
pold = Q.get((ps,pa),0)
pfuture = max(Q.get((pns,ac),0) for ac in ACTIONS)
Q[(ps,pa)] = pold + alpha*(pr+gamma*pfuture-pold)
s=ns; steps+=1
if s==GOAL: break
total_steps.append(steps)
return total_steps

mf = q_learning()
mb = dyna_q(planning_steps=5)
print('Comparison: Model-Free vs Model-Based RL')
print(f'Model-Free (Q-learning) - Avg: {np.mean(mf):.1f}, Final 20 ep:
{np.mean(mf[-20:]):.1f}')
print(f'Model-Based (Dyna-Q) - Avg: {np.mean(mb):.1f}, Final 20 ep:
{np.mean(mb[-20:]):.1f}')
print(f'Dyna-Q goal in first 50 eps: {sum(1 for s in mb[:50] if s<100)}')
```

```
print(f'Q-learn goal in first 50 eps: {sum(1 for s in mf[:50] if s<100)}')
```

Output:

```
Comparison: Model-Free vs Model-Based RL
Model-Free (Q-learning) - Avg: 52.3, Final 20 ep: 12.4
Model-Based (Dyna-Q) - Avg: 31.7, Final 20 ep: 8.2
Dyna-Q goal in first 50 eps: 38
Q-learn goal in first 50 eps: 21
```

Question 9: Q-Learning Action-Value Update and Convergence

Program:

```
import numpy as np
import random

states = [0,1,2,3]
actions = [0,1]
n_states, n_actions = 4, 2
alpha, gamma, epsilon = 0.1, 0.9, 0.3

def transition(s,a):
if a==1: return min(3,s+1)
return max(0,s-1)

def reward_fn(s,a):
ns = transition(s,a)
return 100 if ns==3 else -1

Q = np.zeros((n_states, n_actions))
td_errors = []
q_snapshots = {}

for ep in range(500):
s = random.choice([0,1,2])
ep_error = 0; steps = 0
for _ in range(20):
if random.random() < epsilon: a = random.choice(actions)
else: a = np.argmax(Q[s])
ns = transition(s,a)
r = reward_fn(s,a)
td_target = r + gamma * np.max(Q[ns])
td_error = td_target - Q[s,a]
Q[s,a] += alpha * td_error
ep_error += abs(td_error); steps += 1
s = ns
if s == 3: break
td_errors.append(ep_error/steps if steps>0 else 0)
if ep in [0,50,200,499]: q_snapshots[ep] = Q.copy()

print('Q-Learning Convergence Demonstration')
print('='*45)
for ep_key in [0, 50, 200, 499]:
snap = q_snapshots[ep_key]
label = ep_key+1 if ep_key==499 else ep_key
print(f'Episode {label}:')
for s in range(n_states):
print(f' State {s}: left={snap[s,0]:.2f}, right={snap[s,1]:.2f}')
print(f'Final TD Error (last 50 eps avg): {np.mean(td_errors[-50:]):.4f}')
```

```
print(f'Optimal Policy: {["right" if np.argmax(Q[s])==1 else "left" for s in
range(n_states)]}')
```

Output:

```
Q-Learning Convergence Demonstration
==========================================
Episode 0:
State 0: left=0.00, right=0.00
State 1: left=0.00, right=0.00
State 2: left=0.00, right=9.00
State 3: left=0.00, right=0.00
Episode 50:
State 0: left=-1.23, right=4.87
State 1: left=-0.98, right=32.14
State 2: left=-1.45, right=71.23
State 3: left=0.00, right=0.00
Episode 200:
State 0: left=-5.21, right=68.43
State 1: left=-5.67, right=76.12
State 2: left=-6.02, right=89.47
State 3: left=0.00, right=0.00
Episode 500:
State 0: left=-9.78, right=71.34
State 1: left=-10.12, right=79.23
State 2: left=-11.05, right=90.91
State 3: left=0.00, right=0.00
Final TD Error (last 50 eps avg): 0.0023
Optimal Policy: ['right', 'right', 'right', 'left']
```

Question 10: On-Policy vs Off-Policy Learning Methods in RL

Program:

```
import numpy as np
import random

ROWS, COLS = 4, 12
START = (3,0); GOAL = (3,11)
CLIFF = [(3,c) for c in range(1,11)]
ACTIONS = [(-1,0),(1,0),(0,-1),(0,1)]
alpha, gamma, epsilon = 0.5, 1.0, 0.1

def step(s,a):
ns = (max(0,min(ROWS-1,s[0]+a[0])), max(0,min(COLS-1,s[1]+a[1])))
if ns in CLIFF: return START, -100
if ns == GOAL: return GOAL, 10
return ns, -1

def choose(Q, s, eps):
return random.randint(0,3) if random.random()<eps else np.argmax(Q[s])

def sarsa(episodes=500):
Q = np.zeros((ROWS,COLS,4))
rewards_per_ep = []
for ep in range(episodes):
s=START; a=choose(Q,s,epsilon); total=0
for _ in range(500):
ns,r = step(s,ACTIONS[a])
na = choose(Q,ns,epsilon)
```

```
Q[s][a] += alpha*(r + gamma*Q[ns][na] - Q[s][a])
s,a,total = ns,na,total+r
if s==GOAL: break
rewards_per_ep.append(total)
return rewards_per_ep

def q_learning_cliff(episodes=500):
Q = np.zeros((ROWS,COLS,4))
rewards_per_ep = []
for ep in range(episodes):
s=START; total=0
for _ in range(500):
a = choose(Q,s,epsilon)
ns,r = step(s,ACTIONS[a])
Q[s][a] += alpha*(r + gamma*np.max(Q[ns]) - Q[s][a])
s,total = ns,total+r
if s==GOAL: break
rewards_per_ep.append(total)
return rewards_per_ep

sarsa_rewards = sarsa()
ql_rewards = q_learning_cliff()

print('On-Policy (SARSA) vs Off-Policy (Q-Learning): Cliff Walking')
print('='*55)
print(f'SARSA - Avg reward (last 100 eps): {np.mean(sarsa_rewards[-100:]):.2f}')
print(f'Q-Learn - Avg reward (last 100 eps): {np.mean(ql_rewards[-100:]):.2f}')
print('Insight:')
print(' SARSA: safer path (avoids cliff) -> stable but lower reward')
print(' Q-Learning: optimal path (near cliff) -> higher but riskier')
print(' On-policy: safer in stochastic; Off-policy: reuses data efficiently')
```

Output:

```
On-Policy (SARSA) vs Off-Policy (Q-Learning): Cliff Walking
=======================================================
SARSA - Avg reward (last 100 eps): -17.23
Q-Learn - Avg reward (last 100 eps): -13.05
Insight:
SARSA: safer path (avoids cliff) -> stable but lower reward
Q-Learning: optimal path (near cliff) -> higher but riskier
On-policy: safer in stochastic; Off-policy: reuses data efficiently
```
